

React

Theory

1. React.memo

- **Definition:** Higher-order component that memoizes a component, preventing unnecessary re-renders if props haven't changed.
- **Use case:** Optimizing child components that receive stable props.

2. useCallback

- **Definition:** Returns a memoized version of a callback function.
- **Syntax:**

```
const memoizedFn = useCallback(() => { ... }, [dependencies]);
```

- Prevents function re-creation on every render, useful when passing callbacks to memoized children.

3. useMemo

- **Definition:** Memoizes the result of an expensive calculation.
- **Syntax:**

```
const result = useMemo(() => computeExpensiveValue(num), [num]);
```

- Runs only when dependencies change, avoiding costly recalculations.

4. Optimization Flow

- Parent re-renders → child re-renders unless memoized.
- React.memo + useCallback → prevents child re-render when props are stable.
- useMemo → prevents recalculating heavy logic unnecessarily.

Coding Round Questions

- React.memo → What is React.memo and when should you use it?
- useCallback → Why is useCallback important when passing functions as props?
- useMemo → How does useMemo improve performance?

- Hooks comparison → Compare useMemo and useCallback.
- Performance → How do hooks help optimize React applications?

⚡ Machine Coding Round Challenge

Problem: Product List Optimization

Requirements:

1. Create a parent component ProductList.
2. Maintain a list of products in state.
3. Use useMemo to calculate total price.
4. Use useCallback for a function that filters products.
5. Pass props to a memoized child component ProductCard.

```
import React from "react";
import MemoChild from "./MemoChild";
import { useState } from "react";
import { useCallback } from "react";
import { useMemo } from "react";

const Usememoo = () => {
  console.log("memo parent rendered");

  const [count, setCount] = useState(0);
  const [num, setNum] = useState(1);

  const handleClick = () => {
    setCount((prev) => prev + 1);
  };

  const display = useCallback(() => {
    console.log("Greet");
  }, []);

  const expensiveCalculation = useMemo(() => {
    console.log("expensive");

    return num * 100000000;
  }, [num]);

  return (
    <div>
      <h1>{expensiveCalculation}</h1>
      <h1>Use memo</h1>
    </div>
  );
}
```

```

    <h2>{count}</h2>

    <button onClick={handClick}>Add</button>

    <MemoChild content="memooo" buttonClick={display} />
  </div>
);
};

export default Usememoo;

```

```

import React from "react";

const MemoChild = React.memo(({ content, buttonClick }) => {
  console.log("Memo child rendered");

  buttonClick();

  return (
    <div>
      <h2>{content}</h2>
      { /* <button onClick={buttonClick}>Child count button</button> */ }
    </div>
  );
});

export default MemoChild;

```